JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class GregorianCalendar

java.lang.Object
    java.util.Calendar
        java.util.GregorianCalendar

All Implemented Interfaces:

Serializable, Cloneable, Comparable<Calendar>

---

public class **GregorianCalendar**
extends Calendar

GregorianCalendar is a concrete subclass of Calendar and provides the standard calendar system used by most of the world.

GregorianCalendar is a hybrid calendar that supports both the Julian and Gregorian calendar systems with the support of a single discontinuity, which corresponds by default to the Gregorian date when the Gregorian calendar was instituted (October 15, 1582 in some countries, later in others). The cutover date may be changed by the caller by calling setGregorianChange().

Historically, in those countries which adopted the Gregorian calendar first, October 4, 1582 (Julian) was thus followed by October 15, 1582 (Gregorian). This calendar models this correctly. Before the Gregorian cutover, GregorianCalendar implements the Julian calendar. The only difference between the Gregorian and the Julian calendar is the leap year rule. The Julian calendar specifies leap years every four years, whereas the Gregorian calendar omits century years which are not divisible by 400.

GregorianCalendar implements *proleptic* Gregorian and Julian calendars. That is, dates are computed by extrapolating the current rules indefinitely far backward and forward in time. As a result, GregorianCalendar may be used for all years to generate meaningful and consistent results. However, dates obtained using GregorianCalendar are historically accurate only from March 1, 4 AD onward, when modern Julian calendar rules were adopted. Before this date, leap year rules were applied irregularly, and before 45 BC the Julian calendar did not even exist.

Prior to the institution of the Gregorian calendar, New Year's Day was March 25. To avoid confusion, this calendar always uses January 1. A manual adjustment may be made if desired for dates that are prior to the Gregorian changeover and which fall between January 1 and March 24.

Week Of Year and Week Year

Values calculated for the WEEK_OF_YEAR field range from 1 to 53. The first week of a calendar year is the earliest seven day period starting on

getFirstDayOfWeek() that contains at least getMinimalDaysInFirstWeek() days from that year. It thus depends on the values of getMinimalDaysInFirstWeek(), getFirstDayOfWeek(), and the day of the week of January 1. Weeks between week 1 of one year and week 1 of the following year (exclusive) are numbered sequentially from 2 to 52 or 53 (except for year(s) involved in the Julian-Gregorian transition).

The getFirstDayOfWeek() and getMinimalDaysInFirstWeek() values are initialized using locale-dependent resources when constructing a GregorianCalendar. The week determination is compatible with the ISO 8601 standard when getFirstDayOfWeek() is MONDAY and getMinimalDaysInFirstWeek() is 4, which values are used in locales where the standard is preferred. These values can explicitly be set by calling setFirstDayOfWeek() and setMinimalDaysInFirstWeek().

A *week year* is in sync with a WEEK_OF_YEAR cycle. All weeks between the first and last weeks (inclusive) have the same *week year* value. Therefore, the first and last days of a week year may have different calendar year values.

For example, January 1, 1998 is a Thursday. If getFirstDayOfWeek() is MONDAY and getMinimalDaysInFirstWeek() is 4 (ISO 8601 standard compatible setting), then week 1 of 1998 starts on December 29, 1997, and ends on January 4, 1998. The week year is 1998 for the last three days of calendar year 1997. If, however, getFirstDayOfWeek() is SUNDAY, then week 1 of 1998 starts on January 4, 1998, and ends on January 10, 1998; the first three days of 1998 then are part of week 53 of 1997 and their week year is 1997.

Week Of Month

Values calculated for the WEEK_OF_MONTH field range from 0 to 6. Week 1 of a month (the days with WEEK_OF_MONTH = 1) is the earliest set of at least getMinimalDaysInFirstWeek() contiguous days in that month, ending on the day before getFirstDayOfWeek(). Unlike week 1 of a year, week 1 of a month may be shorter than 7 days, need not start on getFirstDayOfWeek(), and will not include days of the previous month. Days of a month before week 1 have a WEEK_OF_MONTH of 0.

For example, if getFirstDayOfWeek() is SUNDAY and getMinimalDaysInFirstWeek() is 4, then the first week of January 1998 is Sunday, January 4 through Saturday, January 10. These days have a WEEK_OF_MONTH of 1. Thursday, January 1 through Saturday, January 3 have a WEEK_OF_MONTH of 0. If getMinimalDaysInFirstWeek() is changed to 3, then January 1 through January 3 have a WEEK_OF_MONTH of 1.

Default Fields Values

The `clear` method sets calendar field(s) undefined. `GregorianCalendar` uses the following default value for each calendar field if its value is undefined.

Field	Default Value
ERA	AD
YEAR	1970
MONTH	JANUARY
DAY_OF_MONTH	1
DAY_OF_WEEK	the first day of week
WEEK_OF_MONTH	0
DAY_OF_WEEK_IN_MONTH	1
AM_PM	AM
HOUR, HOUR_OF_DAY, MINUTE, SECOND, MILLISECOND	0

Default values are not applicable for the fields not listed above.

Example:

```
// get the supported ids for GMT-08:00 (Pacific Standard Time)
String[] ids = TimeZone.getAvailableIDs(-8 * 60 * 60 * 1000);
// if no ids were returned, something is wrong. get out.
if (ids.length == 0)
    System.exit(0);

 // begin output
System.out.println("Current Time");

// create a Pacific Standard Time time zone
SimpleTimeZone pdt = new SimpleTimeZone(-8 * 60 * 60 * 1000, ids[0]);

// set up rules for Daylight Saving Time
pdt.setStartRule(Calendar.APRIL, 1, Calendar.SUNDAY, 2 * 60 * 60 * 1000);
pdt.setEndRule(Calendar.OCTOBER, -1, Calendar.SUNDAY, 2 * 60 * 60 * 1000);

// create a GregorianCalendar with the Pacific Daylight time zone
// and the current date and time
Calendar calendar = new GregorianCalendar(pdt);
```

```
Date trialTime = new Date();
calendar.setTime(trialTime);

// print out a bunch of interesting things
System.out.println("ERA: " + calendar.get(Calendar.ERA));
System.out.println("YEAR: " + calendar.get(Calendar.YEAR));
System.out.println("MONTH: " + calendar.get(Calendar.MONTH));
System.out.println("WEEK_OF_YEAR: " + calendar.get(Calendar.WEEK_OF_YEAR));
System.out.println("WEEK_OF_MONTH: " + calendar.get(Calendar.WEEK_OF_MONTH));
System.out.println("DATE: " + calendar.get(Calendar.DATE));
System.out.println("DAY_OF_MONTH: " + calendar.get(Calendar.DAY_OF_MONTH));
System.out.println("DAY_OF_YEAR: " + calendar.get(Calendar.DAY_OF_YEAR));
System.out.println("DAY_OF_WEEK: " + calendar.get(Calendar.DAY_OF_WEEK));
System.out.println("DAY_OF_WEEK_IN_MONTH: "
                   + calendar.get(Calendar.DAY_OF_WEEK_IN_MONTH));
System.out.println("AM_PM: " + calendar.get(Calendar.AM_PM));
System.out.println("HOUR: " + calendar.get(Calendar.HOUR));
System.out.println("HOUR_OF_DAY: " + calendar.get(Calendar.HOUR_OF_DAY));
System.out.println("MINUTE: " + calendar.get(Calendar.MINUTE));
System.out.println("SECOND: " + calendar.get(Calendar.SECOND));
System.out.println("MILLISECOND: " + calendar.get(Calendar.MILLISECOND));
System.out.println("ZONE_OFFSET: "
                   + (calendar.get(Calendar.ZONE_OFFSET)/(60*60*1000)));
System.out.println("DST_OFFSET: "
                   + (calendar.get(Calendar.DST_OFFSET)/(60*60*1000)));
System.out.println("Current Time, with hour reset to 3");
calendar.clear(Calendar.HOUR_OF_DAY); // so doesn't override
calendar.set(Calendar.HOUR, 3);
System.out.println("ERA: " + calendar.get(Calendar.ERA));
System.out.println("YEAR: " + calendar.get(Calendar.YEAR));
System.out.println("MONTH: " + calendar.get(Calendar.MONTH));
System.out.println("WEEK_OF_YEAR: " + calendar.get(Calendar.WEEK_OF_YEAR));
System.out.println("WEEK_OF_MONTH: " + calendar.get(Calendar.WEEK_OF_MONTH));
System.out.println("DATE: " + calendar.get(Calendar.DATE));
System.out.println("DAY_OF_MONTH: " + calendar.get(Calendar.DAY_OF_MONTH));
System.out.println("DAY_OF_YEAR: " + calendar.get(Calendar.DAY_OF_YEAR));
System.out.println("DAY_OF_WEEK: " + calendar.get(Calendar.DAY_OF_WEEK));
System.out.println("DAY_OF_WEEK_IN_MONTH: "
                   + calendar.get(Calendar.DAY_OF_WEEK_IN_MONTH));
System.out.println("AM_PM: " + calendar.get(Calendar.AM_PM));
System.out.println("HOUR: " + calendar.get(Calendar.HOUR));
System.out.println("HOUR_OF_DAY: " + calendar.get(Calendar.HOUR_OF_DAY));
System.out.println("MINUTE: " + calendar.get(Calendar.MINUTE));
```

```
System.out.println("SECOND: " + calendar.get(Calendar.SECOND));
System.out.println("MILLISECOND: " + calendar.get(Calendar.MILLISECOND));
System.out.println("ZONE_OFFSET: "
       + (calendar.get(Calendar.ZONE_OFFSET)/(60*60*1000))); // in hours
System.out.println("DST_OFFSET: "
       + (calendar.get(Calendar.DST_OFFSET)/(60*60*1000))); // in hours
```

Since:

1.1

See Also:

TimeZone, Serialized Form

Nested Class Summary

Nested classes/interfaces declared in class Calendar

Calendar.Builder

Field Summary

Fields

Modifier and Type	Field	Description
static final int	AD	Value of the ERA field indicating the common era (Anno Domini), also known as CE.
static final int	BC	Value of the ERA field indicating the period before the common era (before Christ), also known as BCE.

Fields declared in class Calendar

ALL_STYLES, AM, AM_PM, APRIL, areFieldsSet, AUGUST, DATE, DAY_OF_MONTH, DAY_OF_WEEK, DAY_OF_WEEK_IN_MONTH, DAY_OF_YEAR, DECEMBER, DST_OFFSET, ERA, FEBRUARY, FIELD_COUNT, fields, FRIDAY, HOUR, HOUR_OF_DAY, isSet, isTimeSet, JANUARY, JULY, JUNE, LONG, LONG_FORMAT, LONG_STANDALONE, MARCH, MAY, MILLISECOND, MINUTE, MONDAY, MONTH, NARROW_FORMAT, NARROW_STANDALONE, NOVEMBER, OCTOBER, PM, SATURDAY, SECOND, SEPTEMBER, SHORT, SHORT_FORMAT,

SHORT_STANDALONE, SUNDAY, THURSDAY, time, TUESDAY, UNDECIMBER, WEDNESDAY, WEEK_OF_MONTH, WEEK_OF_YEAR, YEAR, ZONE_OFFSET

Constructor Summary

Constructors

Constructor	Description
`GregorianCalendar()`	Constructs a default GregorianCalendar using the current time in the default time zone with the default FORMAT locale.
`GregorianCalendar(int year, int month, int dayOfMonth)`	Constructs a GregorianCalendar with the given date set in the default time zone with the default locale.
`GregorianCalendar(int year, int month, int dayOfMonth, int hourOfDay, int minute)`	Constructs a GregorianCalendar with the given date and time set for the default time zone with the default locale.
`GregorianCalendar(int year, int month, int dayOfMonth, int hourOfDay, int minute, int second)`	Constructs a GregorianCalendar with the given date and time set for the default time zone with the default locale.
`GregorianCalendar(Locale aLocale)`	Constructs a GregorianCalendar based on the current time in the default time zone with the given locale.
`GregorianCalendar(TimeZone zone)`	Constructs a GregorianCalendar based on the current time in the given time zone with the default FORMAT locale.
`GregorianCalendar(TimeZone zone, Locale aLocale)`	Constructs a GregorianCalendar based on the current time in the given time zone with the given locale.

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
void	**add**(int field, int amount)	Adds the specified (signed) amount of time to the given calendar field, based on the calendar's rules.
protected void	**computeFields**()	Converts the time value (millisecond offset from the Epoch) to calendar field values.
protected void	**computeTime**()	Converts calendar field values to the time value (millisecond offset from the Epoch).
boolean	**equals**(Object obj)	Compares this GregorianCalendar to the specified Object.
static GregorianCalendar	**from**(ZonedDateTime zdt)	Obtains an instance of GregorianCalendar with the default locale from a ZonedDateTime object.
int	**getActualMaximum**(int field)	Returns the maximum value that this calendar field could have, taking into consideration the given time value and the current values of the getFirstDayOfWeek, getMinimalDaysInFirstWeek, getGregorianChange and getTimeZone methods.
int	**getActualMinimum**(int field)	Returns the minimum value that this calendar field could have, taking into consideration the given time value and the

		current values of the `getFirstDayOfWeek`, `getMinimalDaysInFirstWeek`, `getGregorianChange` and `getTimeZone` methods.
`String`	**`getCalendarType`**`()`	Returns `"gregory"` as the calendar type.
`int`	**`getGreatestMinimum`**`(int field)`	Returns the highest minimum value for the given calendar field of this `GregorianCalendar` instance.
`final Date`	**`getGregorianChange`**`()`	Gets the Gregorian Calendar change date.
`int`	**`getLeastMaximum`**`(int field)`	Returns the lowest maximum value for the given calendar field of this `GregorianCalendar` instance.
`int`	**`getMaximum`**`(int field)`	Returns the maximum value for the given calendar field of this `GregorianCalendar` instance.
`int`	**`getMinimum`**`(int field)`	Returns the minimum value for the given calendar field of this `GregorianCalendar` instance.
`int`	**`getWeeksInWeekYear`**`()`	Returns the number of weeks in the week year represented by this `GregorianCalendar`.

int	**getWeekYear**()	Returns the week year represented by this GregorianCalendar.
int	**hashCode**()	Generates the hash code for this GregorianCalendar object.
boolean	**isLeapYear**(int year)	Determines if the given year is a leap year.
final boolean	**isWeekDateSupported**()	Returns true indicating this GregorianCalendar supports week dates.
void	**roll**(int field, boolean up)	Adds or subtracts (up/down) a single unit of time on the given time field without changing larger fields.
void	**roll**(int field, int amount)	Adds a signed amount to the specified calendar field without changing larger fields.
void	**setGregorianChange**(Date date)	Sets the GregorianCalendar change date.
void	**setWeekDate**(int weekYear, int weekOfYear, int dayOfWeek)	Sets this GregorianCalendar to the date given by the date specifiers - weekYear, weekOfYear, and dayOfWeek.
ZonedDateTime	**toZonedDateTime**()	Converts this object to a ZonedDateTime that represents the same point on the time-line as this GregorianCalendar.

Methods declared in class Calendar

after, before, clear, clear, clone, compareTo, complete, get, getAvailableCalendarTypes, getAvailableLocales, getDisplayName, getDisplayNames, getFirstDayOfWeek, getInstance, getInstance, getInstance, getInstance, getMinimalDaysInFirstWeek, getTime, getTimeInMillis, getTimeZone, internalGet, isLenient, isSet, set, set, set, set, setFirstDayOfWeek, setLenient, setMinimalDaysInFirstWeek, setTime, setTimeInMillis, setTimeZone, toInstant, toString

Methods declared in class Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

BC

```
public static final int BC
```

Value of the ERA field indicating the period before the common era (before Christ), also known as BCE. The sequence of years at the transition from BC to AD is ..., 2 BC, 1 BC, 1 AD, 2 AD,...

See Also:

Calendar.ERA, Constant Field Values

AD

```
public static final int AD
```

Value of the ERA field indicating the common era (Anno Domini), also known as CE. The sequence of years at the transition from BC to AD is ..., 2 BC, 1 BC, 1 AD, 2 AD,...

See Also:

Calendar.ERA, Constant Field Values

Constructor Details

GregorianCalendar

```
public GregorianCalendar()
```

Constructs a default `GregorianCalendar` using the current time in the default time zone with the default `FORMAT` locale.

GregorianCalendar

```
public GregorianCalendar(TimeZone zone)
```

Constructs a `GregorianCalendar` based on the current time in the given time zone with the default `FORMAT` locale.

Parameters:

`zone` - the given time zone.

Throws:

`NullPointerException` - if `zone` is `null`

GregorianCalendar

```
public GregorianCalendar(Locale aLocale)
```

Constructs a `GregorianCalendar` based on the current time in the default time zone with the given locale.

Parameters:

`aLocale` - the given locale.

Throws:

`NullPointerException` - if `aLocale` is `null`

GregorianCalendar

```
public GregorianCalendar(TimeZone zone,
                         Locale aLocale)
```

Constructs a `GregorianCalendar` based on the current time in the given time zone with the given locale.

Parameters:

`zone` - the given time zone.

`aLocale` - the given locale.

Throws:

`NullPointerException` - if `zone` or `aLocale` is `null`

GregorianCalendar

```
public GregorianCalendar(int year,
                         int month,
                         int dayOfMonth)
```

Constructs a `GregorianCalendar` with the given date set in the default time zone with the default locale.

Parameters:

`year` - the value used to set the `YEAR` calendar field in the calendar.

`month` - the value used to set the `MONTH` calendar field in the calendar. Month value is 0-based. e.g., 0 for January.

`dayOfMonth` - the value used to set the `DAY_OF_MONTH` calendar field in the calendar.

GregorianCalendar

```
public GregorianCalendar(int year,
                         int month,
                         int dayOfMonth,
                         int hourOfDay,
                         int minute)
```

Constructs a `GregorianCalendar` with the given date and time set for the default time zone with the default locale.

Parameters:

`year` - the value used to set the `YEAR` calendar field in the calendar.

`month` - the value used to set the `MONTH` calendar field in the calendar. Month value is 0-based. e.g., 0 for January.

`dayOfMonth` - the value used to set the `DAY_OF_MONTH` calendar field in the calendar.

`hourOfDay` - the value used to set the `HOUR_OF_DAY` calendar field in the calendar.

`minute` - the value used to set the `MINUTE` calendar field in the calendar.

GregorianCalendar

```
public GregorianCalendar(int year,
                         int month,
                         int dayOfMonth,
                         int hourOfDay,
                         int minute,
                         int second)
```

Constructs a GregorianCalendar with the given date and time set for the default time zone with the default locale.

Parameters:

`year` - the value used to set the `YEAR` calendar field in the calendar.

`month` - the value used to set the `MONTH` calendar field in the calendar. Month value is 0-based. e.g., 0 for January.

`dayOfMonth` - the value used to set the `DAY_OF_MONTH` calendar field in the calendar.

`hourOfDay` - the value used to set the `HOUR_OF_DAY` calendar field in the calendar.

`minute` - the value used to set the `MINUTE` calendar field in the calendar.

`second` - the value used to set the `SECOND` calendar field in the calendar.

Method Details

setGregorianChange

```
public void setGregorianChange(Date date)
```

Sets the `GregorianCalendar` change date. This is the point when the switch from Julian dates to Gregorian dates occurred. Default is October 15, 1582 (Gregorian). Previous to this, dates will be in the Julian calendar.

To obtain a pure Julian calendar, set the change date to `Date(Long.MAX_VALUE)`. To obtain a pure Gregorian calendar, set the change date to `Date(Long.MIN_VALUE)`.

Parameters:

`date` - the given Gregorian cutover date.

getGregorianChange

```
public final Date getGregorianChange()
```

Gets the Gregorian Calendar change date. This is the point when the switch from Julian dates to Gregorian dates occurred. Default is October 15, 1582 (Gregorian). Previous to this, dates will be in the Julian calendar.

Returns:

the Gregorian cutover date for this `GregorianCalendar` object.

isLeapYear

```
public boolean isLeapYear(int year)
```

Determines if the given year is a leap year. Returns `true` if the given year is a leap year. To specify BC year numbers, `1 - year number` must be given. For example, year BC 4 is specified as -3.

Parameters:

`year` - the given year.

Returns:

`true` if the given year is a leap year; `false` otherwise.

getCalendarType

```
public String getCalendarType()
```

Returns `"gregory"` as the calendar type.

Overrides:

`getCalendarType` in class `Calendar`

Returns:

`"gregory"`

Since:

1.8

See Also:

Locale extensions,
`Locale.Builder.setLocale(Locale)`,
`Locale.Builder.setUnicodeLocaleKeyword(String, String)`

equals

```
public boolean equals(Object obj)
```

Compares this `GregorianCalendar` to the specified `Object`. The result is `true` if and only if the argument is a `GregorianCalendar` object that represents the same time value (millisecond offset from the Epoch) under the same `Calendar` parameters and Gregorian change date as this object.

Overrides:

`equals` in class `Calendar`

Parameters:

`obj` - the object to compare with.

Returns:

`true` if this object is equal to `obj`; `false` otherwise.

See Also:

`Calendar.compareTo(Calendar)`

hashCode

```
public int hashCode()
```

Generates the hash code for this `GregorianCalendar` object.

Overrides:

`hashCode` in class `Calendar`

Returns:

a hash code value for this object.

See Also:

`Object.equals(java.lang.Object)`,
`System.identityHashCode(Object)`

add

```
public void add(int field,
                int amount)
```

Adds the specified (signed) amount of time to the given calendar field, based on the calendar's rules.

Add rule 1. The value of `field` after the call minus the value of `field` before the call is `amount`, modulo any overflow that has occurred in `field`. Overflow occurs when a field value exceeds its range and, as a result, the next larger field is incremented or decremented and the field value is adjusted back into its range.

Add rule 2. If a smaller field is expected to be invariant, but it is impossible for it to be equal to its prior value because of changes in its minimum or maximum after `field` is changed, then its value is adjusted to be as close as possible to its expected value. A smaller field represents a smaller unit of time. `HOUR` is a smaller field than `DAY_OF_MONTH`. No adjustment is made to smaller fields that are not expected to be invariant. The calendar system determines what fields are expected to be invariant.

Specified by:

`add` in class `Calendar`

Parameters:

`field` - the calendar field.

`amount` - the amount of date or time to be added to the field.

Throws:

`IllegalArgumentException` - if `field` is `ZONE_OFFSET`, `DST_OFFSET`, or unknown, or if any calendar fields have out-of-range values in non-lenient mode.

See Also:

`Calendar.roll(int,int)`,
`Calendar.set(int,int)`

roll

```
public void roll(int field,
                 boolean up)
```

Adds or subtracts (up/down) a single unit of time on the given time field without changing larger fields.

Example: Consider a `GregorianCalendar` originally set to December 31, 1999. Calling `roll(Calendar.MONTH, true)` sets the calendar to January 31, 1999. The `YEAR` field is unchanged because it is a larger field than `MONTH`.

Specified by:

`roll` in class `Calendar`

Parameters:

`field` - the time field.

`up` - indicates if the value of the specified calendar field is to be rolled up or rolled down. Use `true` if rolling up, `false` otherwise.

Throws:

`IllegalArgumentException` - if `field` is `ZONE_OFFSET`, `DST_OFFSET`, or unknown, or if any calendar fields have out-of-range values in non-lenient mode.

See Also:

`add(int,int)`,
`Calendar.set(int,int)`

roll

```
public void roll(int field,
                 int amount)
```

Adds a signed amount to the specified calendar field without changing larger fields. A negative roll amount means to subtract from field without changing larger fields. If the specified amount is 0, this method performs nothing.

This method calls `Calendar.complete()` before adding the amount so that all the calendar fields are normalized. If there is any calendar field having an out-of-range value in non-lenient mode, then an `IllegalArgumentException` is thrown.

Example: Consider a `GregorianCalendar` originally set to August 31, 1999. Calling `roll(Calendar.MONTH, 8)` sets the calendar to April 30, **1999**. Using a `GregorianCalendar`, the `DAY_OF_MONTH` field cannot be 31 in the month April. `DAY_OF_MONTH` is set to the closest possible value, 30. The `YEAR` field maintains the value of 1999 because it is a larger field than `MONTH`.

Example: Consider a `GregorianCalendar` originally set to Sunday June 6, 1999. Calling `roll(Calendar.WEEK_OF_MONTH, -1)` sets the calendar to Tuesday June 1, 1999, whereas calling `add(Calendar.WEEK_OF_MONTH, -1)` sets the calendar to Sunday May 30, 1999. This is because the roll rule imposes an additional constraint: The `MONTH` must not change when the `WEEK_OF_MONTH` is rolled. Taken together with add rule 1, the resultant date must be between Tuesday June 1 and Saturday June 5. According to add rule 2, the `DAY_OF_WEEK`, an invariant when changing the `WEEK_OF_MONTH`, is set to Tuesday, the closest possible value to Sunday (where Sunday is the first day of the week).

Overrides:

`roll` in class `Calendar`

Parameters:

`field` - the calendar field.

`amount` - the signed amount to add to `field`.

Throws:

`IllegalArgumentException` - if `field` is `ZONE_OFFSET`, `DST_OFFSET`, or unknown, or if any calendar fields have out-of-range values in non-lenient mode.

Since:

1.2

See Also:

`roll(int,boolean)`,
`add(int,int)`,
`Calendar.set(int,int)`

getMinimum

```
public int getMinimum(int field)
```

Returns the minimum value for the given calendar field of this `GregorianCalendar` instance. The minimum value is defined as the smallest value returned by the `get` method for any possible time value, taking into consideration the current values of the `getFirstDayOfWeek`, `getMinimalDaysInFirstWeek`, `getGregorianChange` and `getTimeZone` methods.

Specified by:

`getMinimum` in class `Calendar`

Parameters:

`field` - the calendar field.

Returns:

the minimum value for the given calendar field.

See Also:

`getMaximum(int)`, `getGreatestMinimum(int)`, `getLeastMaximum(int)`, `getActualMinimum(int)`, `getActualMaximum(int)`

getMaximum

```
public int getMaximum(int field)
```

Returns the maximum value for the given calendar field of this `GregorianCalendar` instance. The maximum value is defined as the largest value returned by the `get` method for any possible time value, taking into consideration the current values of the `getFirstDayOfWeek`, `getMinimalDaysInFirstWeek`, `getGregorianChange` and `getTimeZone` methods.

Specified by:

`getMaximum` in class `Calendar`

Parameters:

`field` - the calendar field.

Returns:

the maximum value for the given calendar field.

See Also:

`getMinimum(int)`, `getGreatestMinimum(int)`, `getLeastMaximum(int)`, `getActualMinimum(int)`, `getActualMaximum(int)`

getGreatestMinimum

```
public int getGreatestMinimum(int field)
```

Returns the highest minimum value for the given calendar field of this `GregorianCalendar` instance. The highest minimum value is defined as the largest value returned by `getActualMinimum(int)` for any possible time value, taking into consideration the current values of the `getFirstDayOfWeek`, `getMinimalDaysInFirstWeek`, `getGregorianChange` and `getTimeZone` methods.

Specified by:

`getGreatestMinimum` in class `Calendar`

Parameters:

`field` - the calendar field.

Returns:

the highest minimum value for the given calendar field.

See Also:

`getMinimum(int)`, `getMaximum(int)`, `getLeastMaximum(int)`, `getActualMinimum(int)`, `getActualMaximum(int)`

getLeastMaximum

```
public int getLeastMaximum(int field)
```

Returns the lowest maximum value for the given calendar field of this `GregorianCalendar` instance. The lowest maximum value is defined as the smallest value returned by `getActualMaximum(int)` for any possible time value, taking into consideration the current values of the `getFirstDayOfWeek`, `getMinimalDaysInFirstWeek`, `getGregorianChange` and `getTimeZone` methods.

Specified by:

`getLeastMaximum` in class `Calendar`

Parameters:

`field` - the calendar field

Returns:

the lowest maximum value for the given calendar field.

See Also:

`getMinimum(int)`, `getMaximum(int)`, `getGreatestMinimum(int)`, `getActualMinimum(int)`, `getActualMaximum(int)`

getActualMinimum

```
public int getActualMinimum(int field)
```

Returns the minimum value that this calendar field could have, taking into consideration the given time value and the current values of the `getFirstDayOfWeek`, `getMinimalDaysInFirstWeek`, `getGregorianChange` and `getTimeZone` methods.

For example, if the Gregorian change date is January 10, 1970 and the date of this `GregorianCalendar` is January 20, 1970, the actual minimum value of the `DAY_OF_MONTH` field is 10 because the previous date of January 10, 1970 is December 27, 1996 (in the Julian calendar). Therefore, December 28, 1969 to January 9, 1970 don't exist.

Overrides:

`getActualMinimum` in class `Calendar`

Parameters:

`field` - the calendar field

Returns:

the minimum of the given field for the time value of this GregorianCalendar

Since:

1.2

See Also:

getMinimum(int), getMaximum(int), getGreatestMinimum(int), getLeastMaximum(int), getActualMaximum(int)

getActualMaximum

```
public int getActualMaximum(int field)
```

Returns the maximum value that this calendar field could have, taking into consideration the given time value and the current values of the getFirstDayOfWeek, getMinimalDaysInFirstWeek, getGregorianChange and getTimeZone methods. For example, if the date of this instance is February 1, 2004, the actual maximum value of the DAY_OF_MONTH field is 29 because 2004 is a leap year, and if the date of this instance is February 1, 2005, it's 28.

This method calculates the maximum value of WEEK_OF_YEAR based on the YEAR (calendar year) value, not the week year. Call getWeeksInWeekYear() to get the maximum value of WEEK_OF_YEAR in the week year of this GregorianCalendar.

Overrides:

getActualMaximum in class Calendar

Parameters:

field - the calendar field

Returns:

the maximum of the given field for the time value of this GregorianCalendar

Since:

1.2

See Also:

getMinimum(int), getMaximum(int), getGreatestMinimum(int), getLeastMaximum(int), getActualMinimum(int)

isWeekDateSupported

```
public final boolean isWeekDateSupported()
```

Returns true indicating this GregorianCalendar supports week dates.

Overrides:

isWeekDateSupported in class Calendar

Returns:

`true` (always)

Since:

1.7

See Also:

`getWeekYear()`,
`setWeekDate(int,int,int)`,
`getWeeksInWeekYear()`

getWeekYear

```
public int getWeekYear()
```

Returns the week year represented by this `GregorianCalendar`. The dates in the weeks between 1 and the maximum week number of the week year have the same week year value that may be one year before or after the `YEAR` (calendar year) value.

This method calls `Calendar.complete()` before calculating the week year.

Overrides:

`getWeekYear` in class `Calendar`

Returns:

the week year represented by this `GregorianCalendar`. If the `ERA` value is `BC`, the year is represented by 0 or a negative number: BC 1 is 0, BC 2 is -1, BC 3 is -2, and so on.

Throws:

`IllegalArgumentException` - if any of the calendar fields is invalid in non-lenient mode.

Since:

1.7

See Also:

`isWeekDateSupported()`,
`getWeeksInWeekYear()`,
`Calendar.getFirstDayOfWeek()`,
`Calendar.getMinimalDaysInFirstWeek()`

setWeekDate

```
public void setWeekDate(int weekYear,
                        int weekOfYear,
                        int dayOfWeek)
```

Sets this GregorianCalendar to the date given by the date specifiers - weekYear, weekOfYear, and dayOfWeek. weekOfYear follows the WEEK_OF_YEAR numbering. The dayOfWeek value must be one of the DAY_OF_WEEK values: SUNDAY to SATURDAY.

Note that the numeric day-of-week representation differs from the ISO 8601 standard, and that the weekOfYear numbering is compatible with the standard when getFirstDayOfWeek() is MONDAY and getMinimalDaysInFirstWeek() is 4.

Unlike the set method, all of the calendar fields and the instant of time value are calculated upon return.

If weekOfYear is out of the valid week-of-year range in weekYear, the weekYear and weekOfYear values are adjusted in lenient mode, or an IllegalArgumentException is thrown in non-lenient mode.

Overrides:

setWeekDate in class Calendar

Parameters:

weekYear - the week year

weekOfYear - the week number based on weekYear

dayOfWeek - the day of week value: one of the constants for the DAY_OF_WEEK field: SUNDAY, ..., SATURDAY.

Throws:

IllegalArgumentException - if any of the given date specifiers is invalid, or if any of the calendar fields are inconsistent with the given date specifiers in non-lenient mode

Since:

1.7

See Also:

isWeekDateSupported(),
Calendar.getFirstDayOfWeek(),
Calendar.getMinimalDaysInFirstWeek()

getWeeksInWeekYear

```
public int getWeeksInWeekYear()
```

Returns the number of weeks in the week year represented by this GregorianCalendar.

For example, if this GregorianCalendar's date is December 31, 2008 with the ISO 8601 compatible setting, this method will return 53 for the period: December 29, 2008 to January 3, 2010 while

`getActualMaximum(WEEK_OF_YEAR)` will return 52 for the period: December 31, 2007 to December 28, 2008.

Overrides:

`getWeeksInWeekYear` in class `Calendar`

Returns:

the number of weeks in the week year.

Since:

1.7

See Also:

`Calendar.WEEK_OF_YEAR`, `getWeekYear()`, `getActualMaximum(int)`

computeFields

```
protected void computeFields()
```

Converts the time value (millisecond offset from the Epoch) to calendar field values. The time is *not* recomputed first; to recompute the time, then the fields, call the `complete` method.

Specified by:

`computeFields` in class `Calendar`

See Also:

`Calendar.complete()`

computeTime

```
protected void computeTime()
```

Converts calendar field values to the time value (millisecond offset from the Epoch).

Specified by:

`computeTime` in class `Calendar`

Throws:

`IllegalArgumentException` - if any calendar fields are invalid.

See Also:

`Calendar.complete()`, `Calendar.computeFields()`

toZonedDateTime

```
public ZonedDateTime toZonedDateTime()
```

Converts this object to a `ZonedDateTime` that represents the same point on the time-line as this `GregorianCalendar`.

Since this object supports a Julian-Gregorian cutover date and `ZonedDateTime` does not, it is possible that the resulting year, month and day will have different values. The result will represent the correct date in the ISO calendar system, which will also be the same value for Modified Julian Days.

Returns:

a zoned date-time representing the same point on the time-line as this gregorian calendar

Since:

1.8

from

```
public static GregorianCalendar from(ZonedDateTime zdt)
```

Obtains an instance of `GregorianCalendar` with the default locale from a `ZonedDateTime` object.

Since `ZonedDateTime` does not support a Julian-Gregorian cutover date and uses ISO calendar system, the return GregorianCalendar is a pure Gregorian calendar and uses ISO 8601 standard for week definitions, which has `MONDAY` as the `FirstDayOfWeek` and 4 as the value of the `MinimalDaysInFirstWeek`.

`ZoneDateTime` can store points on the time-line further in the future and further in the past than `GregorianCalendar`. In this scenario, this method will throw an `IllegalArgumentException` exception.

Parameters:

`zdt` - the zoned date-time object to convert

Returns:

the gregorian calendar representing the same point on the time-line as the zoned date-time provided

Throws:

`NullPointerException` - if `zdt` is null

`IllegalArgumentException` - if the zoned date-time is too large to represent as a `GregorianCalendar`

Since:

1.8

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which

contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.